9.3 Dictionaries in Python

A **dictionary** is a data structure used to store data in **key-value pairs**.

Syntax

```
dictionary_name = {
    key1: value1,
    key2: value2,
    key3: value3
}
```

1. A dictionary stores data using **keys** and **values**.
2. Each key is connected to one value.
3. Keys are used to access values.
4. Dictionaries are mutable, so values can be changed.
5. Dictionary keys must be unique.
6. Dictionaries are written using curly braces {}.

Flow Chart

```
Create dictionary
   |
   v
Store data as key-value pairs
   |
   v
Use key to access value
   |
   v
Update / add / remove data when needed
```

Example 1

PYTHON CODE

```
student = {
"name": "Aman",
"age": 21,
"course": "Python"
}
print(student)
```

Output: {'name': 'Aman', 'age': 21, 'course': 'Python'}

CodeWithNishchal

Dictionary Structure

```
Dictionary
   |
   v
{
  "name": "Aman",
   key      value

  "age": 21,
   key   value
}
```

Dictionary Properties Table

Property	Meaning	Example
Key-value based	Stores data as pairs	"name": "Aman"
Mutable	Can be changed	student["age"] = 22
Ordered	Keeps insertion order	Python 3.7+
Unique keys	Duplicate keys are not allowed	Last value replaces old value
Fast lookup	Values are accessed using keys	student["name"]
Mixed values allowed	Values can be any data type	string, int, list, tuple, dict

9.3.1 Creating Dictionaries

Syntax

```python
dictionary_name = {
    "key1": "value1",
    "key2": "value2"
}
```

Explanation

1. Dictionaries are created using {}.
2. Keys and values are separated using a colon :.
3. Each pair is separated using a comma.
4. Keys are usually strings, but numbers and tuples can also be used.
5. Values can be any data type.

CodeWithNishchal

PYTHON CODE

```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85.5,
    "is_passed": True
}
print(student)
```

Output: {'name': 'Rahul', 'age': 20, 'marks': 85.5, 'is_passed': True}

Different Ways to Create Dictionaries

Type	Example
Empty dictionary	data = {}
Normal dictionary	student = {"name": "Aman", "age": 21}
Using dict()	student = dict(name="Aman", age=21)
Nested dictionary	students = {"s1": {"name": "Aman"}}
Dictionary with list value	data = {"marks": [80, 90, 85]}
Dictionary with tuple key	points = {(10, 20): "A"}

9.3.2 Dictionary Keys and Values

Syntax

```
dictionary = {
    key: value
}
```

Explanation

1. A **key** is used to identify a value.
2. A **value** is the data stored against the key.
3. Keys must be unique.
4. Values can be duplicate.
5. Keys should be immutable types like string, number, or tuple.

Code with Nishchal

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}

print(student["name"])
```

Output: Aman

Key Rules Table

Rule	Allowed?	Example
String key	Yes	"name": "Aman"
Number key	Yes	1: "One"
Tuple key	Yes	(10, 20): "Point"
List key	No	[1, 2]: "Value"
Duplicate keys	Not useful	Last value is kept

Duplicate Key Example

```python
student = {
    "name": "Aman",
    "name": "Rahul"
}

print(student)
```

Output: {'name': 'Rahul'}

The second value replaces the first value because dictionary keys must be unique.

CodeWithNishchal

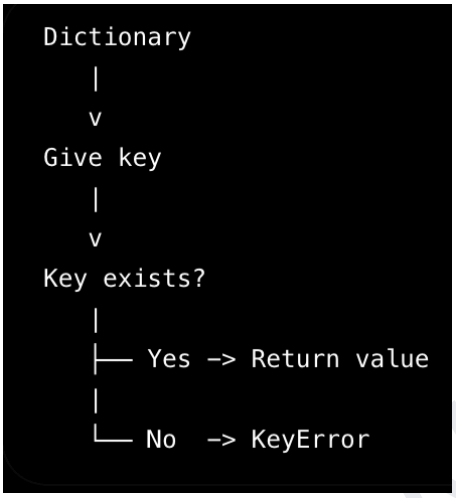
9.3.3 Accessing Values

Syntax

dictionary_name[key]

Explanation

1. Dictionary values are accessed using keys.
2. Unlike lists and tuples, dictionaries are not accessed mainly by index.
3. If the key exists, Python returns its value.
4. If the key does not exist, Python gives KeyError.



Example 1

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}

print(student["name"])
print(student["age"])
```

Output:
Aman
21

CodeWithNishchal

Accessing Table

Code	Meaning	Result
student["name"]	Access name	"Aman"
student["age"]	Access age	21
student["course"]	Access course	"Python"

KeyError Example

```
PYTHON CODE
student = {
"name": "Aman",
"age": 21
}
print(student["marks"])
```

Output:
KeyError: 'marks'
The key "marks" does not exist.

9.3.4 Updating Values

Syntax

dictionary_name[key] = new_value

1. Dictionaries are mutable.
2. Existing values can be updated using keys.
3. If the key already exists, its value is updated.
4. If the key does not exist, a new key-value pair is added.

```
PYTHON CODE
student = {
    "name": "Aman",
    "age": 21
}
student["age"] = 22
student["course"] = "Python"
print(student)
```

Output: {'name': 'Aman', 'age': 22, 'course': 'Python'}

CodeWithNishchal

9.3.5 Dictionary Methods

Dictionary methods are built-in operations used to access, update, remove, copy, and manage dictionary data.

Assume this dictionary:

```python
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
```

Dictionary Methods Table

Method	Purpose	Example	Result / Effect
keys()	Returns all keys	student.keys()	dict_keys(['name', 'age', 'course'])
values()	Returns all values	student.values()	dict_values(['Aman', 21, 'Python'])
items()	Returns key-value pairs	student.items()	dict_items([('name','Aman'),...])
get()	Returns value safely	student.get("name")	"Aman"
update()	Adds or updates data	student.update({"age": 22})	Updates age
pop()	Removes key and returns value	student.pop("age")	Returns 21
popitem()	Removes last inserted pair	student.popitem()	Removes last pair
clear()	Removes all items	student.clear()	{}
copy()	Creates shallow copy	new = student.copy()	New dictionary
setdefault()	Gets value or adds default	student.setdefault("city", "Delhi")	Adds city if missing
fromkeys()	Creates dictionary from keys	dict.fromkeys(["a","b"], 0)	{'a': 0, 'b': 0}

CodeWithNishchal

Method Behavior Table

Method	Changes Original Dictionary?	Returns
keys()	No	View of keys
values()	No	View of values
items()	No	View of key-value pairs
get()	No	Value or default
update()	Yes	None
pop()	Yes	Removed value
popitem()	Yes	Removed key-value pair
clear()	Yes	None
copy()	No	New shallow copy
setdefault()	May change	Existing/default value
fromkeys()	Creates new dictionary	New dictionary

9.3.6 get() Method

Syntax

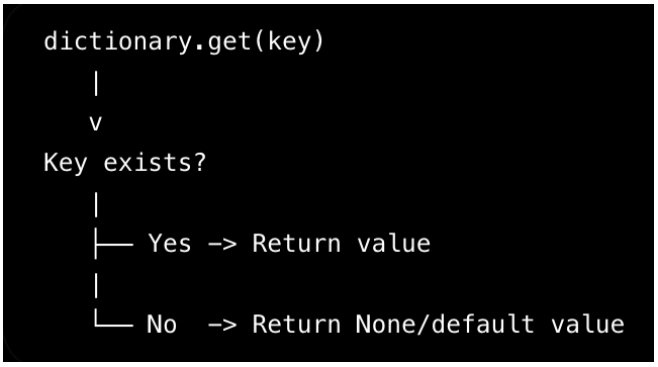
```python
dictionary_name.get(key)
dictionary_name.get(key, default_value)
```

Explanation

1. get() is used to access dictionary values safely.
2. If the key exists, it returns the value.
3. If the key does not exist, it returns None by default.
4. We can also provide our own default value.
5. get() avoids KeyError.

CodeWithNishchal

Flow Chart



Example 1

PYTHON CODE

```
student = {
"name": "Aman",
"age": 21
}
print(student.get("name"))
print(student.get("marks"))
print(student.get("marks", "Not available"))
```

Output:
Aman
None
Not available

[] vs get() Table

Access Method	If Key Exists	If Key Missing
student["name"]	Returns value	Gives KeyError
student.get("name")	Returns value	Returns None
student.get("marks", 0)	Returns value	Returns default value

CodeWithNishchal

9.3.7 Removing Dictionary Items

Syntax

dictionary.pop(key)
dictionary.popitem()
del dictionary[key]
dictionary.clear()

Explanation

1. pop(key) removes a specific key and returns its value.
2. popitem() removes the last inserted key-value pair.
3. del removes a specific key.
4. clear() removes all items.

Removing Items Table

Method / Keyword	Purpose	Example
pop(key)	Removes specific key	student.pop("age")
popitem()	Removes last pair	student.popitem()
del	Deletes specific key	del student["age"]
clear()	Empties dictionary	student.clear()

Example 1

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
student.pop("age")
print(student)
```

Output: {'name': 'Aman', 'course': 'Python'}

deWithNishchal

9.3.8 Looping Through Dictionaries

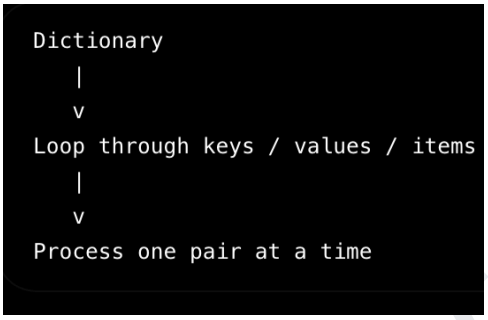
Syntax

```python
for key in dictionary:
    statement
```

```python
for key, value in dictionary.items():
    statement
```

Explanation

1. A dictionary can be looped through using for.
2. By default, looping over a dictionary gives keys.
3. Use values() to loop through values.
4. Use items() to loop through both keys and values.



Example 1

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21,
    "course": "Python"
}
for key, value in student.items():
    print(key, value)
```

Output:

name Aman
age 21
course Python

WithNishchal

9.3.9 Checking Key Membership

Syntax

```
key in dictionary
key not in dictionary
```

Explanation

1. in checks whether a key exists in the dictionary.
2. It checks keys, not values.
3. It returns True or False.

Example 1

```python
student = {
    "name": "Aman",
    "age": 21
}

print("name" in student)
print("marks" in student)
```

Output:
True
False

Membership Table

Code	Meaning
"name" in student	Checks if key exists
"marks" not in student	Checks if key does not exist
"Aman" in student	Checks key, not value

CodeWithNishchal

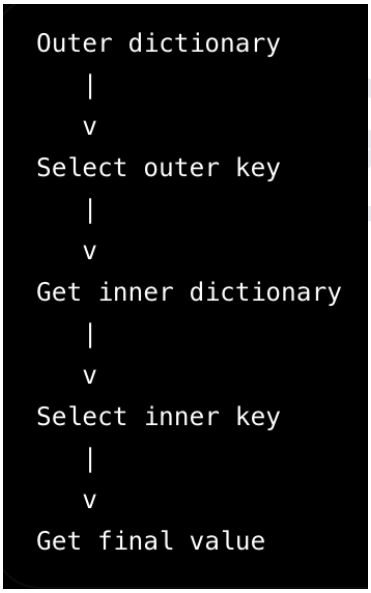
9.3.10 Nested Dictionaries

Syntax

```python
dictionary_name = {
    key1: {
        inner_key1: inner_value1,
        inner_key2: inner_value2
    },
    key2: {
        inner_key1: inner_value1,
        inner_key2: inner_value2
    }
}
```

Explanation

1. A nested dictionary means a dictionary inside another dictionary.
2. It is useful for storing structured data.
3. To access inner values, use multiple keys.
4. Nested dictionaries are common in real-world data like users, students, products, and API responses.



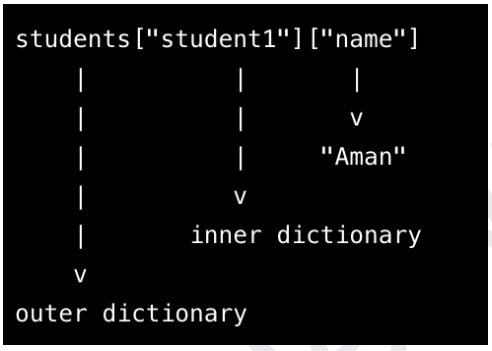
eWith Nishchal

Example 1

PYTHON CODE

```
students = {
    "student1": {
        "name": "Aman",
        "age": 21
    },
    "student2": {
        "name": "Riya",
        "age": 20
    }
}
print(students["student1"]["name"])
print(students["student2"]["age"])
```

Output:
Aman
20



Updating Nested Dictionary

Python

```
students = {
    "student1": {
        "name": "Aman",
        "age": 21
    }
}

students["student1"]["age"] = 22

print(students)
```

Output: {'student1': {'name': 'Aman', 'age': 22}}

devWithNishchal

9.3.11 Dictionary Comprehensions

Syntax

```
new_dict = {key_expression: value_expression for item in sequence}
```

With condition:

```
new_dict = {key_expression: value_expression for item in sequence if condition}
```

Explanation

1. Dictionary comprehension is a short way to create a dictionary.
2. It is similar to list comprehension.
3. It creates key-value pairs using a loop.
4. It can also include conditions.

Flow Chart

Take sequence
|
v
Loop through items
|
v
Create key-value pair
|
v
Store in new dictionary

Example 1

PYTHON CODE

```
numbers = [1, 2, 3, 4]
squares = {number: number * number for number in numbers}
print(squares)
```

Output:

{1: 1, 2: 4, 3: 9, 4: 16}

CodeWithNishchal

Example 2: With Condition

PYTHON CODE

```
numbers = [1, 2, 3, 4, 5, 6]
even_squares = {number: number * number for number in numbers if number % 2 == 0}
print(even_squares)
```

Output: {2: 4, 4: 16, 6: 36}

Dictionary Comprehension Parts

Part	Meaning
number before :	Key
number * number	Value
for number in numbers	Loop
if number % 2 == 0	Optional condition

9.3.12 Merging Dictionaries

Merging means combining two or more dictionaries.

Example 1: Using update()

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21
}
extra = {
    "course": "Python",
    "city": "Delhi"
}
student.update(extra)
print(student)
```

Output: {'name': 'Aman', 'age': 21, 'course': 'Python', 'city': 'Delhi'}

With Nishchal

Example 2: Using |

PYTHON CODE

```
student = {
    "name": "Aman",
    "age": 21
}
extra = {
    "course": "Python",
    "city": "Delhi"
}
result = student | extra
print(result)
```

Output: {'name': 'Aman', 'age': 21, 'course': 'Python', 'city': 'Delhi'}

Same Key During Merge

If both dictionaries have the same key, the second dictionary value wins.

PYTHON CODE

```
a = {"name": "Aman", "age": 21}
b = {"age": 22, "course": "Python"}
result = a | b
print(result)
```

Output: {'name': 'Aman', 'age': 22, 'course': 'Python'}

9.3.13 Copying Dictionaries

Syntax

new_dict = old_dict.copy()

Explanation

1. copy() creates a shallow copy of a dictionary.
2. Direct assignment does not create a new dictionary.
3. Direct assignment makes both variables point to the same dictionary.

Coo hNishchal

Direct Assignment Example

PYTHON CODE

```
student1 = {"name": "Aman", "age": 21}
student2 = student1

student2["age"] = 22
print(student1)
print(student2)
```

Output:
{'name': 'Aman', 'age': 22}
{'name': 'Aman', 'age': 22}

Real Copy Example

PYTHON CODE

```
student1 = {"name": "Aman", "age": 21}
student2 = student1.copy()

student2["age"] = 22
print(student1)
print(student2)
```

Output:

{'name': 'Aman', 'age': 21}

{'name': 'Aman', 'age': 22}

Copying Table

Method	Creates New Dictionary?	Notes
dict2 = dict1	No	Same dictionary reference
dict2 = dict1.copy()	Yes	Shallow copy
dict2 = dict(dict1)	Yes	Shallow copy

Codev Nishchal

9.3.14 Dictionary vs List

Point	List	Dictionary
Data storage	Stores values	Stores key-value pairs
Access by	Index	Key
Brackets	[]	{}
Best for	Ordered collection of values	Labeled/structured data
Example	["Aman", 21]	{"name": "Aman", "age": 21}

9.3.15 Important Dictionary Operations

Operation	Code	Meaning
Create dictionary	data = {"name": "Aman"}	Creates dictionary
Access value	data["name"]	Gets value by key
Safe access	data.get("name")	Avoids KeyError
Update value	data["name"] = "Rahul"	Changes value
Add new pair	data["age"] = 21	Adds key-value pair
Remove key	data.pop("age")	Removes key
Check key	"name" in data	Returns True or False
Get keys	data.keys()	Returns all keys
Get values	data.values()	Returns all values
Get pairs	data.items()	Returns key-value pairs
Merge dictionaries	`dict1	dict2`
Copy dictionary	data.copy()	Creates shallow copy
Empty dictionary	data.clear()	Removes all items

CodeWithNishchal

9.4 Sets in Python

A **set** is a data structure used to store multiple **unique** values.

Syntax

```python
set_name = {value1, value2, value3}
```

Explanation

1. Sets are written using curly braces {}.
2. Set items are separated by commas.
3. Sets store only unique values.
4. Sets are unordered, so items do not have fixed positions.
5. Sets are mutable, so we can add or remove items.
6. Set elements must be immutable/hashable values.

Example 1

PYTHON CODE

```
numbers = {10, 20, 30, 40}
print(numbers)
```

{40, 10, 20, 30} : The output order may look different because sets are unordered.

Set Properties Table

Property	Meaning	Example
Unordered	Items have no fixed index	Cannot use items[0]
Unique values	Duplicate values are removed	{10, 10, 20} becomes {10, 20}
Mutable	Items can be added/removed	items.add(50)
Unindexed	No indexing/slicing	items[1] not allowed
Mixed data allowed	Can store different immutable types	{10, "Aman", True}
Fast membership check	Good for checking existence	10 in numbers

Codevi Nishchal